

Übung 3: Externe Quellen und Excel-Schnittstellen

Zugehöriger Foliensatz: Teil III (Externe Datenquellen und Excel-Schnittstellen).

Vorbereitung

- Arbeiten Sie im RStudio-Projekt `projekt/`. Alle Datenpfade sind relativ als `data/...`
- Starterdatei: `R/lab3_externe_quellen.R` (mit `# TODO-Lücken`).
- Kontrollwerte stehen direkt bei den einzelnen Aufgaben.
- Benötigte Pakete sind installiert: `readxl`, `openxlsx`, `pdftools`, `stringr`, `readr`, `dplyr`, `tidyr`, `DBI`, `RSQLite`.

Datenübersicht

Datei	Inhalt
<code>data/lieferung_excel.xlsx</code>	Blätter Metadaten und Daten. Im Blatt Daten stehen zwei Titelzeilen und eine Leerzeile, die Kopfzeile folgt in Zeile 4 und die Werte ab Zeile 5. Wide-Format (Bereiche x 2018Q1..2024Q4).
<code>data/lieferung_pdf.pdf</code> <code>data/edo_demo.sqlite</code>	Einseitige Tabelle: Bruttowertschöpfung 2024Q4, Mio. EUR. SQLite-DB (EDO-Proxy) mit Tabellen <code>lieferung_2024q4</code> und <code>lieferung_2025q1</code> .
<code>data/lieferung_2024q4.csv</code>	Dieselbe Lieferung als tidy/long CSV (für den Vergleich).

Aufgabe 1: Excel-Lieferung erkunden und einlesen

1. Listen Sie mit `excel_sheets("data/lieferung_excel.xlsx")` die Blattnamen auf.
2. Lesen Sie das Blatt `Metadaten`. Es enthält nur Freitext in einer Spalte.
3. Lesen Sie das Blatt `Daten` so ein, dass die Spaltenüberschriften korrekt erkannt werden.

```
library(readxl)
xl_pfad <- "data/lieferung_excel.xlsx"

blaetter <- NULL # TODO: Blattnamen ermitteln
meta <- NULL # TODO: Blatt "Metadaten" einlesen
wide <- NULL # TODO: Blatt "Daten" mit passendem skip einlesen
```

Hinweise:

- Bei den Metadaten ist `col_names = FALSE` sinnvoll, sonst wird die erste Textzeile zur Kopfzeile.
- Im Blatt `Daten` stehen zwei Titelzeilen und eine Leerzeile. Die Kopfzeile steht in Zeile 4, die Werte beginnen in Zeile 5. Wählen Sie `skip` so, dass Zeile 4 zur Kopfzeile wird. Probieren Sie kurz `skip = 0` und vergleichen Sie die Spaltennamen (`...1`, `...2`) mit dem Ergebnis bei korrektem `skip`.
- Kontrolle: `wide` hat 10 Zeilen und 30 Spalten (`bereich_code`, `bereich_name` plus 28 Zeitspalten).

Aufgabe 2: Wide ins Long-Format bringen

Das Excel ist im Wide-Format. Bringen Sie es ins kanonische Long-Format mit den Spalten `jahr`, `quartal`, `bereich_code`, `bereich_name`, `wert`.

```
library(tidyr); library(dplyr); library(stringr)
```

```
lang <- NULL # TODO: mit pivot_longer() aufbereiten
```

Kontrolle: lang hat 280 Zeilen (10 Bereiche × 28 Quartale).

Aufgabe 3: Auswertung berechnen und nach Excel schreiben

Berechnen Sie die Summe je Bereich über alle vier Quartale 2024 und schreiben Sie das Ergebnis nach output /.

```
library(openxlsx)
dir.create("output", showWarnings = FALSE)

ergebnis <- NULL # TODO: Summe je Bereich für 2024 berechnen
# TODO: ergebnis nach output/ergebnis_summe_2024.xlsx schreiben
```

Hinweise:

- `dir.create("output", showWarnings = FALSE)` legt den Ordner an, ohne zu meckern, falls er schon existiert.
- Optional können Sie zusätzlich ein Workbook mit fetter Kopfzeile bauen (`createWorkbook()`, `addWorksheet()`, `writeData()`, `createStyle()`, `addStyle()`, `setColWidths()`, `saveWorkbook()`).
- Kontrolle: größter Bereich ist BE (896.778 Mio. EUR), kleinster A (35.536 Mio. EUR).

Aufgabe 4: PDF-Tabelle parsen

Die Quartalstabelle liegt nur als PDF vor. Extrahieren Sie die zehn Datenzeilen in eine Tabelle.

```
library(pdftools); library(stringr); library(readr); library(tibble)

seiten <- NULL # TODO: PDF-Text einlesen
zeilen <- NULL # TODO: erste Seite in Zeilen zerlegen
daten_zeilen <- NULL # TODO: Datenzeilen filtern
pdf_tab <- NULL # TODO: Felder zerlegen und als Tibble aufbauen
```

Hinweise:

- `pdf_text()` liefert je Seite eine Zeichenkette. Spalten sind durch mehrere Leerzeichen getrennt, Zeilen durch `\n`.
- Der Regex `^\s*[A-Z]{1,2}\s` fängt genau die Datenzeilen (beginnen mit dem Bereichscode) und lässt Titel- und Leerzeilen weg.
- Die Werte stehen im deutschen Format (9.338,2). `parse_number()` mit passendem `locale` macht daraus eine Zahl.
- Kontrolle: `pdf_tab` hat 10 Zeilen.

Aufgabe 5: EDO-Datenbank abfragen

Sprechen Sie die SQLite-DB (EDO-Proxy) über DBI an.

```
library(DBI)

con <- NULL # TODO: Verbindung öffnen
tabellen <- NULL # TODO: Tabellen auflisten
db_agg <- NULL # TODO: Aggregation per SQL abfragen
db_2024q4 <- NULL # TODO: vollständige Tabelle einlesen
# TODO: Verbindung schließen
```

Hinweise:

- Optional können Sie die Aggregation zusätzlich mit `tbl()` und `collect()` über das `dplyr`-Backend formulieren.
- Schließen Sie die Verbindung am Ende immer mit `dbDisconnect(con)`.
- Kontrolle: 2 Tabellen, die Q4-2024-Summen je Bereich entsprechen den Werten aus dem PDF (Aufgabe 4).

Aufgabe 6: Kurzvergleich DB-Tabelle vs. CSV

Prüfen Sie, ob die DB-Tabelle `lieferung_2024q4` mit `data/lieferung_2024q4.csv` übereinstimmt.

```
library(readr); library(dplyr)

csv_2024q4 <- NULL # TODO: CSV einlesen
db_s <- NULL # TODO: DB-Tabelle sortieren
csv_s <- NULL # TODO: CSV-Tabelle sortieren
gleich <- NULL # TODO: beide Tabellen vergleichen
```

Hinweise:

- Sortieren Sie beide Tabellen identisch, bevor Sie vergleichen.
- `all.equal()` meldet `TRUE` bei Gleichheit, sonst eine Liste der Unterschiede.
- Kontrolle: beide haben 280 Zeilen und sind inhaltlich identisch (`gleich == TRUE`).